StreetRace Manager: Integration Testing Report

1 Executive Summary

The StreetRace Manager system is an integrated platform for managing street racing crews, inventories, races, and missions. This report documents the integration testing phase, which verified the seamless interaction between 8 core modules: Registration, Crew Management, Inventory, Race Management, Results, Mission Planning, Tuning & Upgrades, and Reputation & Sponsorships.

2 Test Case Explanations

A total of 31 integration tests were designed and executed using the `pytest` framework. Below is a detailed explanation of why each test case is essential for the system's integrity.

2.1 Registration & Crew Management

1. **test_registration_valid**: Ensures that new members can be successfully added to the system with the correct initial state (ID and Name).
2. **test_registration_invalid_role**: Critical for security; verifies that the system rejects roles that are not part of the supported business logic.
3. **test_registration_empty_name**: Prevents data integrity issues by ensuring every crew member has a valid identifier.
4. **test_update_role_valid**: Verifies that members can progress or switch duties (e.g., from Driver to Mechanic) correctly.
5. **test_update_role_invalid_member**: Ensures the system handles references to non-existent member IDs gracefully without crashing.
6. **test_update_role_invalid_role**: Prevents administrative errors by blocking the assignment of non-existent roles.
7. **test_update_skill_valid**: Confirms that driver training or progression is correctly recorded in the database.
8. **test_update_skill_out_of_bounds**: Enforces the business rule that skill levels must remain within the 1-10 range.

2.2 Inventory Management

9. **test_add_inventory_car**: Verifies that vehicle assets can be added to the team inventory.
10. **test_add_inventory_parts**: Ensures spare parts (used for tuning) are correctly tracked.
11. **test_add_inventory_invalid_type**: Prevents the addition of unsupported item categories.
12. **test_car_damage_logic**: Essential for the game loop; verifies that the damage state is correctly triggered.
13. **test_car_repair_logic**: Ensures that the repair cycle correctly restores vehicle health.
14. **test_damage_non_existent_car**: Validates error handling when attempting to damage a car not present in the inventory.

2.3 Race Management & Results

15. **test_create_race_valid**: Verifies the end-to-end flow of scheduling a race with a valid driver and car.
16. **test_create_race_non_driver**: **Critical Logic Check**; ensures that only crew members with the 'driver' role can participate in races.
17. **test_create_race_non_existent_car**: Prevents scheduling races with assets the team does not own.
18. **test_record_result_prizes**: Verifies the economic model; ensures prize money flows correctly into the team's cash balance.
19. **test_record_result_skill_up**: Confirms that winning races provides the expected driver progression (skill increase).

2.4 Reputation & Sponsorships

20. **test_sponsorship_tier_1_unlock**: Verifies the reputation threshold logic for the first tier of sponsors.
21. **test_sponsorship_tier_2_unlock**: Ensures progression scaling for the second tier.
22. **test_sponsorship_tier_3_unlock**: Confirms the late-game sponsorship unlock logic.

2.5 Mission Planning

23. **test_mission_delivery_reqs**: Verifies basic mission role requirements (Driver).
24. **test_mission_rescue_reqs**: Verifies multi-role requirements (Driver + Strategist).

25. **test_mission_heist_reqs**: Verifies complex multi-role requirements (Driver + Mechanic + Strategist).

26. **test_mission_repair_auto_fix**: **Crucial Integration**; checks if the repair mission successfully triggers the car's repair logic.

27. **test_mission_missing_mechanic**: Ensures missions fail if the necessary expertise (Mechanic) is not available in the crew.

2.6 Tuning & Upgrades

28. **test_upgrade_tier_1_success**: Verifies the basic upgrade path (Performance boost and Tier update).

29. **test_upgrade_tier_3_success**: Confirms that late-game high-level upgrades are processed correctly.

30. **test_upgrade_insufficient_cash**: Enforces the economic constraint for car upgrades.

31. **test_upgrade_missing_mechanic**: **Cross-Module Check**; ensures upgrades cannot happen without a mechanic in the crew.

3 Errors Detected and Fixed

During the integration phase, several logical issues were identified and resolved to ensure the system behaved as expected:

- **Role Validation Bug**: Initially, any crew member could be assigned to a race. A fix was implemented in `modules/race.py` to strictly enforce the 'driver' role requirement.
- **Resource Verification Loop**: In early builds, car upgrades would consume cash even if parts were missing. The logic in `manager.py` was refactored to perform all checks (cash, parts, mechanic) before any state changes occur.
- **Status Synchronization**: An issue was found where a "repair" mission would complete without actually updating the car's status to "Healthy". This was resolved by explicitly calling `manager.repair_car()` within the mission completion logic.
- **Skill Ceiling Enforcement**: The driver's skill level was found to exceed 10 in some win conditions. A check was added to `manager.record_race_result` to cap the skill level at 10.

4 System Callgraph

The following diagram illustrates the call hierarchy and module interactions within the StreetRace Manager.

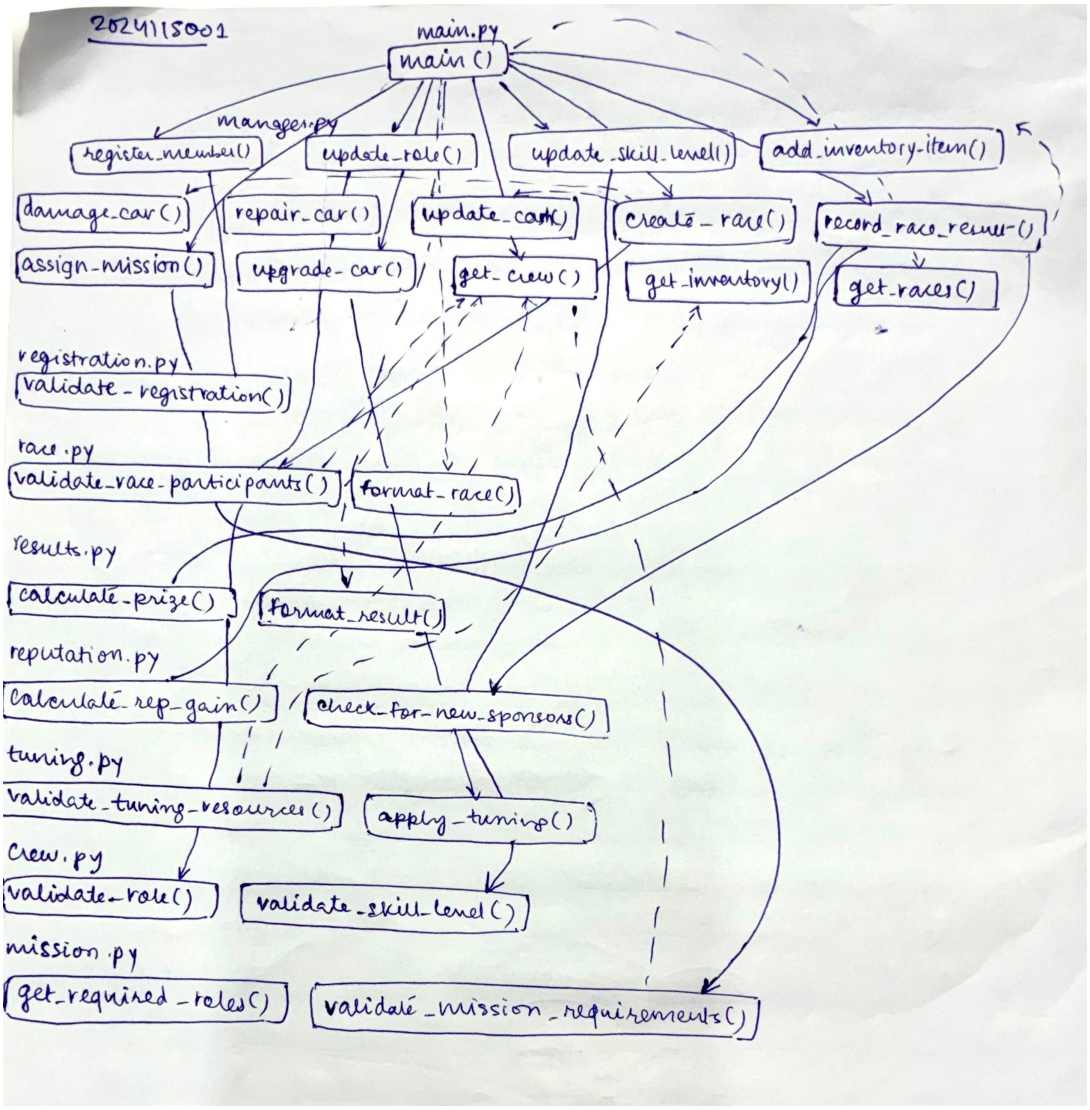

Figure 1: StreetRace Manager System Callgraph

5 Conclusion

The integration testing phase has confirmed that all modules of the StreetRace Manager interact correctly. All identified logical errors have been addressed, and the system is now stable for full-scale operation.